

G00 · 오리엔테이션과 기호 사전

G00 · 오리엔테이션과 기호사전

이 권은 “테스트타임 학습과 신경 메모리” 시리즈 전체의 **입구이자참조 사전**이다. 여기서는 이 계열 논문 여섯 편이 공유하는 **단하나의 핵심 아이디어** (“메모리 = test-time 에 학습시키는 작은 모델”) 를 먼저 그림으로 잡고, 이 시리즈를 읽는 법을 안내한 다음, 나머지 아홉권에서 계속 튀어나올 **모든 기호를 평이한 뜻으로 한 표에 모은다**. 뒤 권을 읽다가 기호가 막히면 언제든지 이 권으로 돌아오면 된다.

대상 독자는 transformer inference 와 서빙 시스템은 잘 알지만 (KV cache, pre-fill/decode, FlashAttention, roofline, batching) **수식과 training 은 거의 안 보는** 엔지니어다. 목적은 증명이 아니라, 이 계열을 **시스템 모델링에 쓸 수준**으로 이해하는 것이다.

0.1 이 시리즈의 한문장

이 시리즈 전체를 한 문장으로 줄이면 이렇다.

핵심 메모리는 저장 버퍼가 아니라, 읽는 도중에도 계속 학습되는 작은신경망이다. token 하나가 들어올 때마다 이 작은 모델을 gradient step 한번으로 갱신한다. 여섯 논문은 전부 “이 갱신을 어떻게 하느냐” 의변주다.

당신이 아는 세계에서 forward 는 weights 를 읽기만 한다. weights 는 여러 request 가 공유하고, 유일하게 변하는 상태는 append-only KV cache 다. prefill/decode 비대칭, continuous batching, paged attention 은 전부 이 한전제 (“weights 는 얼어 있다”) 의 파생물이다.

이 계열은 그 전제를 **한 군데에서 깬다**. sequence layer 안에 작은신경망을 하나 심어 두고, decode 가 token 을 하나 처리할 때마다 그 신경망의 weights 를 살짝 갱신한다. 즉 token 하나를 처리하는 것이 그 작은 모델의 forward → loss → backward → update 한 사이클이다.

비유 KV cache 에 “압축 codec” 이 하나 달렸다고 상상하라. 새 token 이 올 때마다 codec 은 그 정보를 자기 안에 밀어 넣는다. 그런데 그 밀어 넣는 연산이 단순복사 (append) 가 아니라 **gradient step** 이다. codec 은 매 token 마다 “내가 지금까지 배운 걸로 이 token 을 잘 맞히나?” 를 시험하고, 틀린 만큼자기 weights 를 고친다. 이 codec 이 바로 “메모리” 다.

이것은 fine-tuning 이 아니다. label 도 없고 훈련 job 도 없다. layer forward 안에 내장된 연산일 뿐이다. 원형 논문 (Sun et al. 2024, arXiv:2407.04620) 은 이를 “hidden state 가 곧 모델 그 자체이고, update rule 이 self-supervised learning 의 한 step” 이라고 요약한다.

0.2 두 세계: 얼어 있는 weights 와 움직이는 weights

이 계열을 이해하는 첫걸음은 **두 종류의 weights 를 구분하는** 것이다. 이름을 정확히 붙여 두면 나머지 권이 훨씬 쉬워진다.

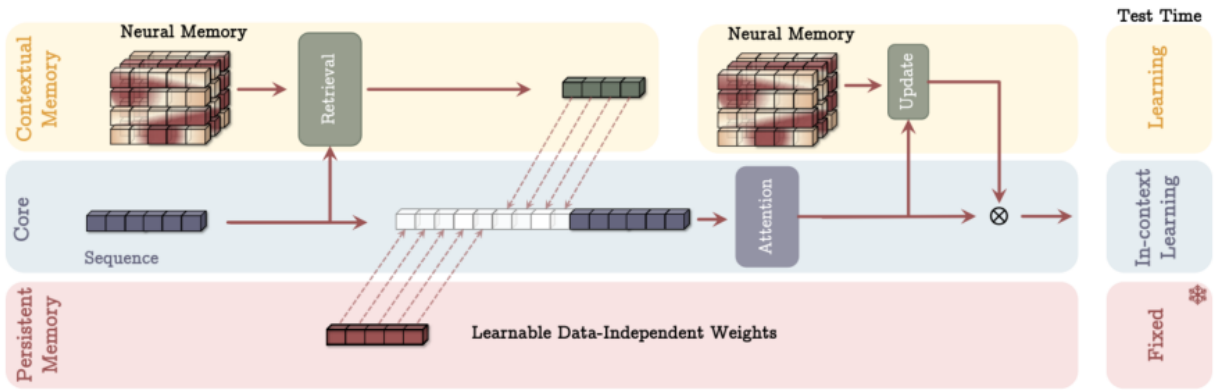


그림 0-1. Titans 의 개요: 세 갈래 (core / long-term memory / persistent memory) 로 구성된 Memory-as-Context(MAC) 구조. 가운데의 long-term memory 가 “test-time 에 학습되는 작은 모델” 이다. 출처: Behrouz et al., Titans (arXiv:2501.00663) Fig.2 — 원저자 그림.

- **fast weights** (W) — token 마다 갱신되는 **작은네트워크**의 weights. request 가 시작될 때 초깃값 W_{init} 에서 출발하고, request 가 끝나면 버려지거나 세션상태로 관리된다. 이게 “메모리 = 작은 모델” 의 그 모델이다.
- **slow weights** (Θ) — 당신이 아는 통상적인 parameter. projection 행렬, backbone, gate 를 만드는 head 등. pretraining 이 끝나면얼어붙고, 서빙 중에는 읽기 전용이다.

기호 풀이 W = fast weights(inner-loop 에서 token 마다 변하는 작은 모델의 상태). W_{init} = 그 fast weights 의 초깃값 (pretraining 으로 미리학습됨). Θ (대문자 세타) = slow weights 전체 (projection·gate 생성기·backbone). 이 **대문자 구분 (W vs Θ)** 이 이시리즈에서 가장 중요한 표기 약속이다.

즉 이 계열이 폐기하는 불변식은 “모든 weights가 얼어 있다” 이고, 서빙 중에도 여전히 유지되는 불변식은 “ Θ 는 얼어 있다” 이다. 움직이는 것은 오직 W 뿐이다. (예외: G08 Sleep 권은 이 규칙마저주기적으로 깨고 slow weights 를 offline 으로 갱신한다.)

시스템 모델링 관점 당신의 멘탈 모델에서 상태는 딱 하나, KV cache 였다. 이제 상태가 하나 더 생긴다: per-request fast-weight state W . 이 W 는 고정 크기 행렬 (또는 작은 MLP 의 weights) 이고, context 길이와 무관하게 크기가 일정하다. 대신 request 마다 개별적으로 존재하므로 “weights 는 공유된다” 는 batching 전제를 깨뜨린다. 새 cache class 가 하나 생겼다고 보면 된다 — sizing, checkpoint/restore, eviction 을 다시 설계해야 하는 대상 (→ G09 시스템 모델링 권).

0.3 모든 sequence 모델은 이미메모리였다

Titans 논문 (arXiv:2501.00663) § 2 의 출발점은 이것이다: 어떤 recurrent sequence 모델이든 메모리의 write/read 연산 쌍으로 다시 쓸 수 있다.

$$W_t = f(W_{t-1}, x_t) \quad (\text{write})$$

$$y_t = g(W_t, x_t) \quad (\text{read})$$

기호 풀이 $x_t = t$ 번째 token 의 표현 (입력 벡터). $W_t =$ token t 까지 처리한 뒤의 메모리 상태. $y_t =$ layer 가 그 token 에서 내보내는 출력. $f =$ write 규칙 (새 정보를 메모리에 반영), $g =$ read 규칙 (메모리에서 답을 꺼냄). $t =$ token 인덱스 (1 부터).

직관 위 두 줄은 “메모리에 쓰고, 메모리에서 읽는다” 를 수식으로 옮긴것뿐이다. 새 token 이 오면 write 로 상태 $W_{t-1} \rightarrow W_t$ 를 갱신하고, 그상태로 read 해서 출력 y_t 를 만든다.

당신이 아는 두 극단이 이 틀의 양 끝이다.

- **softmax attention + KV cache** = 압축하지 않는 메모리. 지금까지의 key/value 를 하나도 버리지 않고 목록으로 쌓아 두고, attention 으로 통째로 훑어 읽는다. 무손실이고 용량이 사실상 무한이지만, 상태 크기와 token 당 read 비용이 context 길이 L 에 비례해자란다 ($O(L)$).
- **linear attention / linear-RNN state** = 압축하는메모리. 고정 크기 행렬 W 에 정보를 계속 겹쳐 쓴다. 비용은고정이지만, 서로 다른 정보가 같은 자리에 겹쳐 쓰이면서 회수가오염된다 (이 오염을 crosstalk 이라 부른다).

비유 KV cache 는 모든 영수증을 상자에 다 모아 두는 것이다 (무손실, 대신 상자가 계속 커짐). linear-RNN state 는 가계부 한 장에 총액만계속 덮어쓰는 것이다 (크기 고정, 대신 세부가 몽개짐). 이 계열의 출발질문은 “덮어쓰는 규칙을 똑똑하게 만들면 같은 크기 가계부로 얼마나손실을 줄일 수 있나?” 이다.

시스템 모델링 관점 모든 sequence layer 는 네 칸짜리 답으로 요약된다: (1) **state** 가무엇인가 (KV 목록 vs $d \times d$ 행렬), (2) **write** 는어떻게 (append vs 덮어쓰기 vs gradient step), (3) **read** 는어떻게 (q 로 목록 훑기 vs 행렬 곱), (4) **retention** 은어떻게 (안 지움 vs 조금씩 지움). 뒤 권들은 매 논문마다 이 네 칸을채운다. 모델링할 때 이 네 칸만 물으면 어떤 변종이든 위치를 잡을 수 있다.

0.4 기준 수식 (M): 이 시리즈의 뼈대 한줄

이 시리즈에는 기준 수식 (master update) 하나가 있다. 여섯 논문이전부 이 한 줄의 부품을 바꾼 것이다. 지금은 유도가 아니라읽는 법만 익히면 된다.

$$S_t = \beta_t S_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t), \quad W_t = \alpha_t W_{t-1} + S_t$$

읽기는 $y_t = \mathcal{M}(q_t; W_t)$ 한 줄로 끝난다. 기호가 많아 보이지만 전부 당신이 아는 개념이다.

기호 풀이 $\ell(W; k_t, v_t) = \text{inner loss}$. 기본형은 $\|\mathcal{M}(k_t; W) - v_t\|_2^2$, “key k_t 를 넣었을 때 메모리가 value v_t 를 잘 뱉나?” 의 per-token 오차 측정기. $k_t, v_t, q_t =$ 이 token 의 key / value / query(전부 slow weights 가 만든 벡터). $\nabla_W \ell =$ 그 오차가 가장 커지는 방향 (gradient); 실제 write 신호는 반대 부호 ($-\eta_t \nabla_W \ell$). $S_t =$ gradient 를 누적하는 buffer, 곧 **momentum**. $\eta_t =$ inner learning rate(이번 token 을 얼마나 세게 반영할지). $\beta_t =$ momentum decay(과거 방향의 관성을 얼마나 유지할지). $\alpha_t \in [0, 1] =$ retention gate(옛 메모리를 얼마나 남길지; 1=전부 유지, 0=완전 소거). $\mathcal{M}(\cdot; W) =$ read 함수 (상태 W 로 답을 계산). 시간 첨자 t 가 붙은 gate 는 상수가 아니라 token 마다 계산되는 값이다.

이제 각 부품을 한 줄 직관으로.

- **surprise** = 얼마나 틀렸나. $\nabla_W \ell$ 은 “지금 메모리가 이 token 을 얼마나 못 맞히나” 의 방향과 크기다. Titans 는 이것을 **surprise** 라 부른다. 많이 틀릴수록 (놀랄수록) 세게 쓴다.
- **momentum** = 최근 놀란 방향의 관성. $S_t = \beta_t S_{t-1} - \eta_t \nabla_W \ell$ 은 “이번에 놀란 방향” 에 “직전까지 놀라 온 방향” 을 관성으로 더한 것이다. 이 줄은 당신이 아는 scan/prefix-sum kernel 이 처리하는 선형 recurrence 와 같은대수다.
- **retention** = 조금씩 지우기. $W_t = \alpha_t W_{t-1} + S_t$ 의 α_t 는 옛 상태를 얼마나 남길지 정한다. $\alpha_t < 1$ 이면 매 token 마다 옛 메모리가 조금씩 바뀐다 — 이것이 학습된 **eviction** 이다.

직관 이 한 줄은 결국 “gradient descent + momentum + weight decay 를 token 스트림 위에서 돌리면, 그게 곧 sequence layer 다” 라는 뜻이다. 당신이 optimizer 로 알던 것 (SGD·momentum·weight decay) 이 이제 layer 내부에서 token 마다 돈다.

시스템 모델링 관점 이 한 줄에서 토큰당 무슨 일이 벌어지는지 세어 보자. state 는 세 덩어리다: 메모리 W (예: $d \times d$ 행렬 $\rightarrow d^2$ 개 숫자), momentum buffer S (같은 크기 d^2), 그리고 read 를 위한 임시값. **토큰당 연산** 은 (1) read 1 회 ($\mathcal{M}(k_t; W)$), (2) backward 1 회 (surprise $\nabla_W \ell$ 계산 — 대략 forward 의 2 배), (3) S 갱신 (read-modify-write), (4) W 갱신 (read-modify-write). 즉 KV cache 가 append 한 번이던 자리에, 여기서는 **state 전체를 읽고-고치고-쓰는 사이클**이 돈다. recurrence(순차 의존성) 는 $S_t \leftarrow S_{t-1}$, $W_t \leftarrow W_{t-1}$ 두 곳에 있다 — 이게 decode 를 순차적으로 묶는 지점이고, chunk 병렬화 ($\rightarrow$ G02, G06) 가 공략하는 대상이다.

0.5 두 개의 loop: 누가 무엇을 학습하는가

training 을 안 해 본 독자의 첫 질문은 늘 같다. “test time 에 학습한다면, 그 learning rate(η_t) 는 누가 정하나?” 답은 두 개의 loop 로 나뉜다.

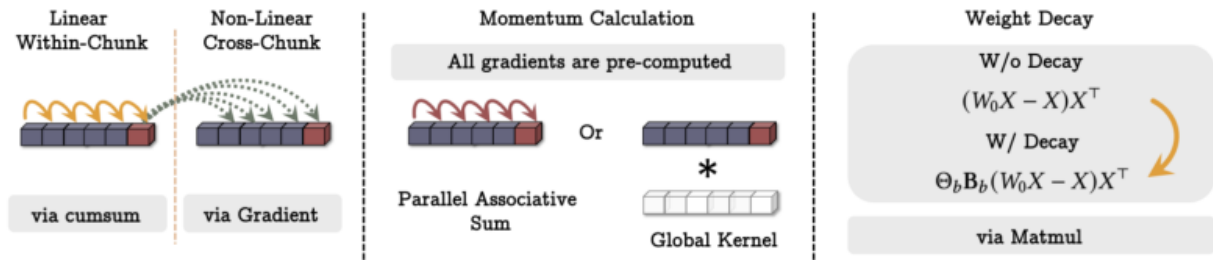


그림 0-2. 신경 메모리의 학습은 matmul로 병렬화할 수 있다 — chunk 단위 병렬 write의 개념도. inner loop의 “mini-batch”가 여러 token(chunk)임을 보여 준다. 출처: Behrouz et al., Titans (arXiv:2501.00663) Fig.1 — 원저자 그림.

- **inner loop** — token마다 fast weights W 를 갱신하는 과정. 위 (M)식이 바로 inner loop다. request 안에서, decode 도중에 돈다.
- **outer loop** — pretraining. slow weights Θ 를 학습하는 과정. 여기서 Θ 에는 projection, gate를 만드는 head, 그리고 초기 메모리 W_{init} 까지 포함된다.

핵심은 이렇다. gate들 $(\eta_t, \beta_t, \alpha_t)$ 은 상수가 아니라 **slow weights가 만드는 token의 함수**다. 예를 들어 $\eta_t = \eta(x_t; \Theta)$ — “이 token을 얼마나 세게 쓸지”를 Θ 가 token을 보고 매번 계산한다. 그리고 그 Θ 는 outer loop(pretraining)가 inner loop 전체를 관통해 backpropagate하며 학습한다.

핵심 “누가 학습하는가?”의 답은 **항상 outer loop(Θ)**다. inner loop는 배운 정책을 test time에 **실행**할 뿐이다. “무엇을 세게 쓰고 무엇을 잊을지”라는 정책 자체는 pretraining에서 이미 배워둔다.

비유 slow weights Θ 는 **운전 교습소에서 배운 운전습관**이다 (언제 세게 밟고 언제 잊을지의 정책). fast weights W 는 **오늘 실제 주행 중 머릿속에 쌓이는 경로 기억**이다. 습관은 교습 (pretraining)에서 완성되고, 오늘의 기억은 주행 (inference)마다 새로 쌓였다 사라진다.

주의 여기서 “batch 축”의 역할을 **sequence 축이 대신한다**. 보통 training에서 mini-batch는 여러 example이지만, inner loop의 “mini-batch”는 여러 **token**(= chunk)이다. 뒤 권에서 “이건 example 축인가 token 축인가?”를 매번 물어야 한다 — 이 계열 최대의 혼동 지점이다.

0.6 기호 대사전

이 절이 이 권의 심장이다. 뒤 아홉 권에서 쓰는 기호를 **평이한 뜻**으로 모았다. 막히면 여기로 돌아오라. 기호는 성격별로 나눠 풀이 박스에 담는다.

기호 풀이 — **입력 · 차원 · 인덱스**. $x_t = t$ 번째 입력 token 벡터. X = 시퀀스 전체 행렬 (행이 token). L = sequence 길이 (토큰수). d = model 차원, d_k/d_v =

key/value 차원, d_h = 메모리 hidden 차원. t, τ = token 인덱스 (1 부터). n = chunk 인덱스 (0 부터). i, j = 보조 인덱스 (window 안 위치 등). L_{layer} = 네트워크 layer 수 (깊이) — sequence 길이 L 과 혼동 금지.

기호 풀이 — 두 loop 의 대상. W_t = fast weights = 메모리상태 (inner loop, token 마다 변함). deep memory 면 $W = \{W_1, W_2, \dots\}$ (여러 층). W_{init} = 메모리초깃값 (W_0 ; pretraining 으로 학습됨). Θ = slow weights 전체 (outer loop). $\theta^{(\ell)}$ = level ℓ 의 parameter (계층형 문맥, $\rightarrow$ G07). $\mathcal{M}(\cdot; W)$ = 메모리모델 (함수). $\mathcal{M}^*$ = inner 문제의 최적해 (argmin) — read 표기가 아니라 “이상적 답” 표기.

기호 풀이 — key/value/query 와 projection. k_t, v_t, q_t = 이 token 의 key / value / query 벡터 ($k_t = W_K x_t$ 식으로 만듦). W_K, W_V, W_Q = 각각을 만드는 projection 행렬 (slow weights, Θ 의 일부). K, V, Q = 그것들을 행으로 쌓은 행렬. y_t = layer 출력. P = persistent memory tokens (N_p 개; task 마다 고정된, 학습된 “메모 쪽지” , $\rightarrow$ G03).

기호 풀이 — loss·gradient·surprise. $\ell(W; k_t, v_t)$ = inner loss (per-token; 기본형 $\|\mathcal{M}(k_t; W) - v_t\|_2^2$). $\mathcal{L}$ = outer(task) loss (next-token prediction 등). 소문자 ℓ 과 대문자 $\mathcal{L}$ 로 두 loop 를 구분한다. L (첨자 없는 대문자, 함수 자리) = attentional bias (inner loss 의 family, $\rightarrow$ G04). g_t = 문맥상 gradient; 두 loop 가 한 식에 있으면 g_t^{in} (inner) / g_t^{out} (outer). momentary surprise = $g_t^{\text{in}} = \nabla_W \ell$. u_t = Local Surprise Signal (= $\nabla_{y_t} \mathcal{L}$, $\rightarrow$ G07). δ_ℓ = backprop 오차 (layer ℓ).

기호 풀이 — optimizer 상태·게이트 (가장 자주 쓰임). S_t = inner momentum buffer (“과거 surprise”). m_t, h_t = outer optimizer 의 1 차/2 차 moment (Adam 류, $\rightarrow$ G01). η_t = inner learning rate (token 마다 계산되는 gate); 첨자 없는 η = outer learning rate (상수). β_t = inner momentum decay (gate). $\alpha_t \in [0, 1]$ = retention gate ($\alpha_t W_{t-1}$; 1=전부 유지, 0=소거). λ = outer weight decay. $\gamma_{t,i} \in [0, 1]$ = window gate (Omega rule 의 in-context pruning, $\rightarrow$ G05). δ_t (시간첨자) = Huber threshold ($\rightarrow$ G04; layer 첨자 δ_ℓ 과 구분). NS_κ = Newton-Schulz κ 회반복 (semi-orthogonalization; $\kappa = 5$ 가 표준, $\rightarrow$ G05).

기호 풀이 — chunk·window·계층. C = chunk 크기 (병렬화 knob 이자, 이 계열에서는 계산되는 함수까지 바꾸는 semantic knob). c (소문자) = window 길이 (Omega rule 의 objective 파라미터) — C 와 반드시 구분. $\xi(t, C)$ = chunk 시작 offset (= $C \lfloor (t-1)/C \rfloor$; gradient 를 어느 상태에서 재나의 anchor). $W^g, W^{l(i)}$ = global / i 번째 local 메모리 상태 ($\rightarrow$ G06 TNT). $L_s^{(i)}$ = local 메모리 reset 주기. Π_t = Q-K projection 행렬 ($\rightarrow$ G06). f_ℓ = update frequency (level ℓ 이 얼마나 자주 갱신되나, $\rightarrow$ G07). $C^{(\ell)}$ = level ℓ 의 chunk 크기.

기호 풀이 — feature map·기타. ϕ_p = 차수 p polynomial feature map ($\rightarrow$ G05). ϕ^* = exponential feature map ($\exp(q^\top k) = \phi^*(q)^\top \phi^*(k)$; softmax attention 을 “무한 용량 메모리” 로 보는 다리). $\sigma(\cdot)$ = activation (GELU/SiLU). $\odot$

= elementwise 곱. $vk^\top$ = outer product(write의 기본 단위). $\|\cdot\|_F$ = Frobenius norm. 표준 deep memory 는 2-layer residual MLP: $\mathcal{M}(z; W) = z + W_1\sigma(W_2z)$.

아래 표는 위 풀이 중 가장 자주 마주칠 기호만 한눈에 모은것이다.

기호	한 줄 뜻	어느 loop / 성격
W_t	fast weights = 메모리 상태 (token 마다 변함)	inner, 상태
Θ	slow weights(projection·gate·backbone)	outer, 얼어있음
W_{init}	메모리 초깃값 (학습됨)	outer 가 학습, inner 가사용
k_t, v_t, q_t	이 token 의 key / value / query	입력 파생
$\ell / \mathcal{L}$	inner loss / outer(task) loss	소문자=inner, 대문자=outer
$\nabla_W \ell$	surprise = 얼마나 틀렸나 (방향)	inner, write 신호
S_t	momentum buffer = 과거 놀란 방향의 관성	inner, 상태
η_t	inner learning rate(얼마나 세게 쓰나)	gate(token 마다)
β_t	momentum decay(관성 유지율)	gate(token 마다)
α_t	retention gate(얼마나 남기나; 1=유지)	gate = 학습된 eviction
C / c	chunk 크기 / window 길이	병렬화 knob / objective 파라미터
$\mathcal{M}(\cdot; W)$	read 함수 (상태로 답 계산)	inner, 함수

주의 — 기호 충돌 3 가지. (1) η 는 첨자가붙으면 (η_t) inner 의 token 별 gate, 안 붙으면 (η) outer 의상수다. **첨자 유무 자체가 정보.** (2) L 은 sequence 길이, ℓ (소문자) 은 inner loss, L (함수 자리) 은 attentional bias —자리로 구분한다. (3) C (chunk) 와 c (window) 는 다른 물건이다. 원논문들은 같은 대상을 서로 다른 기호로 써서 충돌이 잦는데, 이 시리즈는 위통일 표기로 고정한다 (원문 표기와의 대응은 각 권 안에 실린다).

0.7 Rosetta: inference 어휘 ↔ 이 라인의어휘

당신이 이미 아는 서빙 개념으로 이 계열을 번역한 사전이다. 셋째 열의 성격표시가 중요하다 — **동일** (같은 걸 다르게 부름) / **유비** (비슷하나 새 요소 있음) / **△ 차이**가 **논점** (직관을그대로 옮기면 틀림).

inference 세계 (당신의 어휘)	이 라인의 어휘	대응의 성격
KV cache	non-parametric 메모리 상태; softmax attention = 무한 용량의 associative memory	동일 — attention 은 압축하지 않는 메모리
KV cache append	메모리 write (outer-product write 가 최근접; delta rule 은 append 가 아니라 덮어쓰기)	유비 + 차이: cache 는 append-once, W 는 read-modify-write
attention lookup ($q \cdot K$)	메모리 read $y_t = \mathcal{M}(q_t; W_t)$	동일 추상화 ($k \rightarrow v$ 연상)
linear-RNN state($d \times d$)	matrix memory = 고정 크기 lossy 압축메모리	동일
prefill	큰 chunk 의 병렬 write(압축)	유비 (정확)
decode	$C = 1$ 의 per-token online write + read	유비 — 단 backward 가 decode 에 들어온다 는 게 신 세계
FlashAttention tiling	chunkwise-parallel training 의 chunk	△ 차이 : tiling 은 bit-exact, chunk 는 함수 자체를 바꾸는 근사
GEMM shape 감각	$\nabla_W \ell$ 의 outer-product 구조: $dW = (\text{오차}) k^\top$ 는 rank- C GEMM	동일 — 훈련 수식을 GEMM shape 로 읽기
scan/prefix-sum kernel	momentum 의 associative scan	동일
roofline / arithmetic intensity	chunk 크기 C 가 arithmetic intensity 를 정함	동일 도구, 새 독립변수 C
batching(공유 weights 전제)	per-request fast-weight state 가 그 전제를 깨뜨림 → grouped-GEMM decode	△ 차이가 논점
paged KV cache / session cache	per-session weight state — 새로운 cache class	유비 → G09 의 주제
cache eviction policy	retention gate = 학습된 eviction	유비 (정확)
optimizer state(경험 없음)	훈련의 “register file / accumulator” : m_t, h_t	도입용 유비 (→ G01)
weight update 없음 (추론 불변식)	폐기되는 불변식 — 이 라인의 정의적 특징	차이 그 자체
sequence 축	훈련의 batch 축을 sequence 축이 대신함 (inner mini-batch = chunk)	△ 최대 혼동 지점
서빙 fleet 의 백그라운드 job	sleep phase = 주기적 offline consolidation(“weights 의 background compaction”)	유비 → G08

한계 △ 표시 세 줄은 직관을 그대로 옮기면 **틀린다**. 특히 “FlashAttention tiling $\approx$ chunk” 는 위험한 유비다. tiling 은 결과가 bit-exact 인 재배열이지만, 이 계열의 chunk 는 chunk 안 모든 gradient 를 **chunk 시작 상태에서** 계산하는 근사라서, chunk 크기 C 가 **계산되는 함수 자체를 바꾼다**. 그래서 C 는 단순 성능 knob 이아니라

품질에 영향을 주는 semantic hyperparameter 다 ($\rightarrow$ G02, G06).

0.8 이 시리즈를 읽는법

이 시리즈는 열 권이다. 서로 의존성이 있으므로 아래 순서로 읽는 것이 가장쌌다.

권	제목	무엇을 얻나
G00	오리엔테이션 · 기호 사전 (이 권)	핵심 한 문장 + 기호대사전 + Rosetta
G01	학습 기초	backward pass, SGD·momentum·AdamW·Muon 을 (상태, update, 비용) 객체로
G02	빌딩 블록	associative memory, linear attention, chunkwise 병렬화, 기준 수식 조립
G03	Titans	기준 수식 (M) 의 수립: surprise 가 write 강도를정한다
G04	Miras	$\ell(\text{attentional bias})$ 과 retention 항을 바꾼통일 framework
G05	Atlas	write 를 window 목적함수 (Omega rule) 로, capacity 를 feature map 으로
G06	TNT	chunk 크기의 speed - quality 충돌: hierarchical memory + reset
G07	Nested Learning	2-level $\rightarrow$ K-level: 각자 주파수로갱신되는 nested memory
G08	Sleep	wake/sleep lifecycle: 빠른 메모리 $\rightarrow$ 느린 weights 로 consolidation
G09	시스템 모델링 관점	위 전부를 state budget·roofline·serving 으로 환산

핵심 G00 $\rightarrow$ G01 $\rightarrow$ G02 는 기초다. 여기서 기준 수식 (M) 의 부품이 손에잡히면, G03~G08 의 각 논문은 “처음 보는 수식” 이 아니라 “(M) 의 부품을 바꾼 재배선” 으로 읽힌다. 각 논문 권은 세로로 읽으면계보, 가로로 읽으면 “(M) 의 어느 부품을 바꿨나” 의 지도다.

여섯 논문이 (M) 의 어느 부품을 건드리는지 한눈에:

논문 (권)	(M) 에서 바꾸는 것	한 줄
Titans (G03)	(M) 그 자체를 수립	GD+momentum+weight decay 가 test-time write 규칙
Miras (G04)	ℓ 과 retention 항	기존 모델 전부를 하나의 online-optimizer framework 로
Atlas (G05)	ℓ 의 범위 (c) 와 inner optimizer	window 목적함수+ feature map 으로 capacity 확장
TNT (G06)	훈련 경제학 (chunk 크기 C)	큰 C 훈련과 작은 C 서빙을 양립
Nested Learning (G07)	층위 ($2 \rightarrow K$ level), update frequency	nested associative memory 스펙트럼
Sleep (G08)	lifecycle(train/test 경계)	wake 흡수 + sleep consolidation

0.9 첫 back-of-envelope: 왜 이게 시스템문제인가

마지막은 숫자다 (아래 산수는 예시이며 특정 논문의 수치 주장이 아니다). 전형적 GQA 한 layer(KV head 8, head 차원 128, bf16) 에서 KV cache 는 token 당 $2 \times 8 \times 128 \times 2B = 4 \text{ KiB}$ 다. 128K context 면 layer 당 512 MiB, 32-layer 면 request 하나가 **16 GiB** 다. 반면 같은 구성의 matrix memory(head 당 128×128) 는 32 layer 전체가 **8 MiB** 이고 **context 길이와 무관하게 고정**이다.

시스템 모델링 관점 이게 이 계열이 long-context 에서 갖는 구조적 지렛대다: state 가 context 에 따라 자라지 않는다. 대신 네 가지를 지불한다 — (1) decode 안에 backward·update 가 들어옴 (token 당 연산 약 3 배, backward $\approx$ forward 의 2 배), (2) state 전체의 read-modify-write 트래픽, (3) shared-weight batching 이 깨져 grouped-GEMM decode 가 됨, (4) chunk 를 키우면 함수가 바뀌어 훈련품질이 떨어짐. 어느 쪽이 이득인지는 **context 길이**에 달렸다.

직관 위 그림 한 장이 이 시리즈의 시스템적 동기다. “KV 나 신경 메모리나” 는이기고 지는 문제가 아니라 **crossover 문제**다 — context 길이 S^* 를 기준으로 갈린다. 이 계열 전체는 “그 crossover 를 어디로 옮기고, 그 대가를 어떻게 관리하나” 를 다룬다. 정밀 계산과 roofline 배치는 G09 가 담당한다.

요약 - 이 시리즈의 한 문장: 메모리는 test-time 에 token 마다 gradient step 으로 학습되는 작은 모델이다. write 가 곧 학습 step 이다. - weights 는 두 종류: **fast weights** W (token 마다 변함, inner loop) 와 **slow weights** Θ (얼어 있음, outer loop). “누가 학습하나?” 의 답은 항상 **outer loop**(Θ). - 기준 수식 (M) 은 **GD + momentum +**

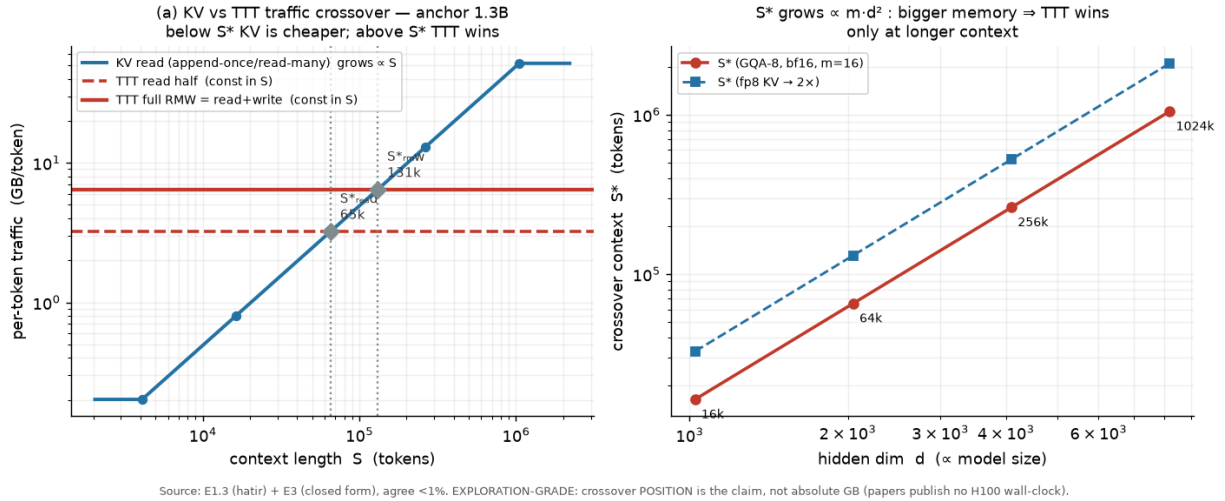


그림 0-3. KV cache 와 TTT(신경 메모리) 의 token 당 트래픽 crossover. 짧은 context 에서는 KV 가 싸고, 어떤 임계 길이 S^* 를 넘으면 고정크기 메모리가 이긴다. 오른쪽: 메모리가 클수록 (모델이 클수록) 그 임계길이 가 더 뒤로 밀린다. 출처: 저자 자체 실험 (hatir E1.3 + E3 closed form) — exploration-grade, crossover 의 위치가 주장.

weight decay 를 token 스트림에서 돌린것이다: $\text{surprise}(\nabla_w \ell) =$ 얼마나 틀렸나, $\text{momentum}(S_t) =$ 놀란 방향의 관성, $\text{retention}(\alpha_t) =$ 조금씩 지우기. 여섯 논문은 (M) 의 부품을 바꾼 것이다. - 시스템 관점에서 이 계열은 상태를 **context 와 무관하게 고정** 시키는 대신, decode 안 backward-state RMW 트래픽 ·batching 붕괴 ·chunk 품질을 지불한다. KV vs 신경 메모리는 **crossover 문제**다. - 이 권은 참조 사전이다 — 기호가 막히면 § 0.6 으로, 서빙 어휘 번역이 막히면 § 0.7 로 돌아오라.

다음 권 예고. G01(학습 기초) 은 이 권이 블랙박스로 남긴 “gradient 가 무엇이고 어떻게 계산되나” 를 연다 — backward pass 를 밑바닥부터 세우고 SGD·momentum·AdamW·Muon 을 각각 (상태, update, 비용) 을 가진 객체로 정의하면, (M) 의 부품이 손에 잡힌다.